Engineering · Distributed Systems · Financial Infrastructure · Event Sourcing

Building a Lending Platform Across Four Markets: Correctness at Scale

How we designed Samsung Finance+'s lending infrastructure to guarantee financial correctness under distributed failure — and why the hardest problems were not throughput, but consistency, auditability, and the operational reality of third-party payment rails.

Utkarsh Mehrotra · Chief Engineer, Samsung Electronics · Samsung Finance+

4	0	T+0	1×
Active markets TH · ID · PH · IN	Core changes per new geography	Settlement from T+2 manual	Every payment event processed, guaranteed

Background

When Samsung Finance+ began expanding its device financing product across Southeast Asia and India, the engineering team hit a problem that compounds silently until it becomes unmanageable: every new market was a separate engineering project. Thailand had different regulatory requirements around loan tenures and repayment schedules. Indonesia required different currency handling and local payment rail integrations. The Philippines had its own collection workflows and delinquency thresholds. India brought an entirely different compliance surface.

The natural instinct — branch the codebase, override per market — is a trap. By the time a third market is live, the fourth requires an audit of every service before a single transaction can be processed. The bugs that surface in market three are invisible until they hit market four with different timing, different rail behaviour, and a different regulatory consequence.

> The architectural question was not "how do we support four markets." It was "how do we design a system where a financial correctness failure in one market cannot propagate to another, and where the fifth market costs us nothing to add." Those two constraints — isolation and extensibility — shaped every decision.

System Requirements

R1 · Market isolation and extensibility	Market-specific behaviour — repayment schedules, currency, collection thresholds, rail integrations — must be expressible as configuration. A bug in Thailand's EMI calculation must not touch Indonesia's loan state. A new market must require zero changes to core service logic.

R2 · Financial correctness under partial failure	A repayment event processed twice must produce exactly one ledger entry. A loan state transition interrupted mid-execution must be resumable without double-booking. This is a correctness requirement — the consequence of failure is a financial statement error or a regulatory finding.
R3 · Full audit replay	Every loan's current state must be deterministically reconstructible from its event history. Regulators in all four markets require reconstructible audit trails — not log approximations, but the ability to replay every state transition from origination to closure.
R4 · Third-party rail resilience	Payment rail providers across four markets have materially different availability and delivery SLAs. The platform must remain financially consistent during provider outages, delayed settlement signals, and duplicate event delivery — without manual intervention.

Key Design Decisions

1. Config-driven market abstraction: versioned contracts, not runtime flags

The v1 instinct for multi-market configuration is a feature flag system: check the market ID at runtime, branch on it. This works for two markets. By four markets, the branching is pervasive. By eight, it is the dominant maintenance burden. The failure mode is not complexity — it is silent misconfiguration.

The design we landed on makes market behaviour entirely external to the services that consume it, with two additions over a naive config store: a schema registry with versioned contracts, and canary rollout for config changes. The canary hash runs on loanId rather than a random seed — ensuring the same loan always sees the same config version during rollout, preventing split-brain within a single saga execution.

```
// Config schema versioning — services declare the version they support
data class MarketConfig(
    val schemaVersion       : Int,
    val marketId            : String,
    val currencyCode        : String,
    val roundingRule        : RoundingRule,
    val settlementWindowMs  : Long,    // per-rail SLA tolerance
    val delinquencyPolicy   : DelinquencyPolicy,
    val railAdapterId       : String,
    val stateTableExtensions: List<StateExtension>
)

// Canary rollout — deterministic per loan, not per request
fun resolveConfig(marketId: String, loanId: String): MarketConfig {
    val canary = configStore.getCanary(marketId)
    if (canary != null && stableHash(loanId) % 100 < canary.rolloutPct) {
        return canary.config
    }
    return configStore.getStable(marketId)
}
```

2. Transactional outbox: decoupling rail integration from event publication

The naive payment integration architecture: the rail adapter receives a settlement event, writes to the loan database, then publishes to the event bus. This is a dual-write. If the database write succeeds and the event bus publish fails, the loan state is updated but no downstream service knows it happened. The system is silently inconsistent.

The transactional outbox pattern resolves this. The rail adapter writes the payment event to an outbox table in the same database transaction as the loan update. A separate relay process reads unprocessed outbox rows and publishes them to the event bus. The loan state update and the event publication are now atomic — either both happen, or neither does.

```
// Transactional outbox — atomic loan update + event publication
fun handleSettlementEvent(event: SettlementEvent) = db.transaction {
    loanRepo.markPaymentReceived(event.loanId, event.amount, event.settledAt)
    outboxRepo.insert(OutboxEntry(
        aggregateId    = event.loanId,
        eventType      = "PAYMENT_SETTLED",
        payload        = event.toJson(),
        idempotencyKey = event.providerRef,
        createdAt      = Instant.now()
    ))
    // Transaction commits. Relay picks up outbox row and publishes.
    // If relay fails, it retries — loan state is already correct.
}
```

3. Distributed saga: financial correctness across service boundaries

Processing a repayment spans multiple services: reconciliation verifies the amount, the ledger service records the entry, the loan service transitions the state, and the delinquency service rescores. In a monolith, these are sequential steps in a single database transaction. In a distributed system, they are separate network calls that can fail at any point.

A local database transaction cannot span service boundaries. The distributed alternative is a saga — a sequence of local transactions, each publishing an event that triggers the next step. If a step fails, compensating transactions undo the preceding steps. The saga store persists progress so the orchestrator can resume after a crash.

> The compensation for a ledger write is not a delete — it is an accounting contra-entry. Deleting a ledger row violates audit requirements in every market we operate in. The compensating transaction produces a new row that exactly offsets the original, preserving the complete financial history. This distinction — between technical rollback and accounting reversal — is one that most distributed systems literature ignores because it is domain-specific.

4. Event sourcing: loan state as a deterministic function of its history

In a conventional architecture, the loan record is mutable state. The current balance, status, and payment schedule are columns that get updated in place. Given the current state, you cannot reconstruct how you got there. When a regulator asks for every state transition for a loan from origination to closure, the answer requires log reconstruction — a forensic exercise that is imprecise by construction.

We model the loan as an event-sourced aggregate. The append-only event log is the source of truth. The current loan state — the projection — is derived by replaying the event stream. The audit replay function is load-bearing infrastructure: when a regulator queries a disputed loan in Indonesia at a specific timestamp, replayAt() produces a deterministic answer from the immutable event log. The event log is stored with WORM retention.

```
// Event-sourced loan aggregate — state is a pure function of events
fun apply(loan: Loan, event: LoanEvent): Loan = when (event) {
    is PaymentSettled -> loan.copy(
        outstandingBalance = loan.outstandingBalance - event.amount,
        state = if (loan.outstandingBalance == event.amount) LoanState.CLOSED
                else loan.state,
        version = event.sequenceNumber
    )
    is OverdueThresholdCrossed -> loan.copy(
        state = LoanState.DELINQUENT, version = event.sequenceNumber
    )
}

// Audit replay — reconstruct any point-in-time state
fun replayAt(loanId: String, asOf: Instant): Loan =
    eventStore.streamEvents(loanId)
        .takeWhile { it.occurredAt <= asOf }
        .fold(Loan.empty(loanId), ::apply)
```

The Latency Budget Model

A lending lifecycle system is not a real-time scoring pipeline — the latency SLA is measured in seconds for reconciliation, not milliseconds. But the principle of owned per-stage latency budgets applies equally. Without explicit budgets, latency regressions are invisible until they cascade: a slow ledger write delays saga completion, which delays delinquency scoring, which delays automated collection workflows during peak repayment periods.

Stage	P99 Budget	Notes
Rail adapter → outbox write	50ms	Local DB transaction, no network hop
Outbox relay → event bus	200ms	At-least-once, async — retried on failure
Reconciliation step	300ms	Circuit breaker fires at 250ms
Ledger write step	300ms	Accounting contra-entry on compensation
Loan state transition	250ms	Optimistic lock — retry on version conflict
Delinquency rescore	250ms	Settlement gate checked first
Audit event write	async	Fire-and-forget — never on critical path
Total P99	< 2s	Settlement → fully consistent state

A Non-Obvious Production Problem: The Delayed Settlement Cascade

The failure mode that cost the most debugging time was not a component failure — it was an emergent interaction between three independently correct subsystems.

The sequence: a customer repays via GCash. GCash sends an acknowledgement immediately — payment accepted, pending settlement. Normally, settlement completes in seconds. Under peak load on a Friday evening, settlement is delayed by 47 minutes. During those 47 minutes, the delinquency scoring pipeline runs its scheduled sweep. The loan's overdue age crosses the threshold. The pipeline triggers an automated collection workflow — an SMS to a customer who has already paid.

Each subsystem was individually correct. The problem was a missing synchronisation point: the delinquency pipeline had no visibility into the settlement window state maintained by the reconciliation service. The fix introduces a settlement window gate as an explicit synchronisation contract between the two pipelines.

```
// Settlement window gate — explicit synchronisation between pipelines
fun evaluateCollectionTrigger(loan: Loan): CollectionDecision {
    val config = configStore.resolve(loan.marketId)
    val pending = reconciliationSvc.getPendingPayment(loan.loanId)

    if (pending != null) {
        val age = Duration.between(pending.acceptedAt, Instant.now())
        return when {
            age < config.settlementWindowMs ->
                CollectionDecision.SUPPRESS          // within SLA window
            age < config.settlementWindowMs * 3 ->
                CollectionDecision.ESCALATE_TO_REVIEW // past SLA, not auto-collect
            else ->
                CollectionDecision.INITIATE_COLLECTION // provider has materially failed
        }
    }
    return delinquencySvc.score(loan, config.delinquencyPolicy).toCollectionDecision()
}
```

Chaos Engineering as a Design Requirement

The delayed settlement cascade was discovered in production. It should have been discovered in staging. The reason it was not: the staging environment did not simulate adversarial provider behaviour. After this incident, we made chaos engineering a design requirement — every rail integration ships with a fault injection profile exercised in staging before any production deployment.

Fault scenario	What it tests	Target subsystem
Settlement delayed N minutes	Settlement window gate suppresses collection triggers	Delinquency pipeline
Duplicate settlement event	Idempotency key prevents double ledger entry	Outbox relay + reconciliation
Out-of-order events	Saga handles unexpected event sequence	Saga orchestrator

Fault scenario	What it tests	Target subsystem
Ledger service timeout mid-saga	Compensation reverses reconciliation; retry completes	Saga compensating txns
Config store unavailable	Services fall back to last-known-good config; alert fires	Config resolution layer
Audit write latency spike	Saga P99 unaffected — audit is async	Latency budget isolation

Summary of Key Decisions

Decision	Option chosen	Rationale
Config versioning	Schema registry + canary rollout	Config changes carry the same production risk as code — they require the same deployment discipline
Event publication	Transactional outbox	Eliminates dual-write; loan DB write and event publication are atomic or neither occurs
Multi-service correctness	Saga + compensating transactions	Distributed transactions are unavailable; saga provides atomic-or-compensated across service boundaries
Ledger compensation	Accounting contra-entry, not deletion	Regulatory audit requirements in all four markets prohibit deleting financial records
Loan state model	Event sourcing + append-only log	Deterministic audit replay; mutable state cannot satisfy regulatory reconstruction requirements
Concurrent write safety	Optimistic concurrency via event version	Makes concurrent transition races structurally detectable; no silent corruption
Collection trigger safety	Settlement window gate	Pipeline interaction failures require explicit synchronisation contracts between independently-scheduled subsystems
Resilience validation	Fault injection per rail integration	Adversarial provider behaviour is not discoverable through standard integration testing

What We Would Do Differently

01 **Ship the saga store and DLQ tooling before going live — not after.**
Saga failures and unreconcilable outbox events surfaced in production before the operational tooling to inspect, replay, and resolve them was ready. In a financial system, a stuck saga represents a loan in an inconsistent state. The replay and resolution tooling should be a launch gate, not a follow-up.

02

Define config schema contracts before the first service ships against them.
The compatibility shim added after the v1 to v2 config schema migration was a three-week engineering engagement that should have been unnecessary. Schema-first design — agree the contract, version it, write the migration path — before any consumer is deployed costs two days. Retrofitting it costs twenty.

03

Model pipeline interactions as first-class design artifacts.
The delayed settlement cascade was caused by an unmodelled interaction between two independently correct pipelines. We now produce a pipeline interaction map as part of every service design review, identifying all shared state and the synchronisation contract at each boundary.

04

Instrument saga step latency and compensation rate as primary metrics.
We added saga observability reactively — after a compensation storm from a degraded ledger service was invisible in our dashboards for 18 minutes. The compensation rate is one of the highest-signal metrics in a saga-based system: a spike indicates either downstream service degradation or a correctness problem in the saga definition itself.

Conclusion

The dominant constraint in financial infrastructure is not throughput — it is correctness under failure. A lending platform that processes repayments correctly 99.9% of the time and double-books 0.1% of the time is not a 99.9% correct system. It is a system with a correctness bug that surfaces on a schedule determined by transaction volume.

The architecture described here — transactional outbox, saga orchestration with compensating transactions, event sourcing with deterministic replay, settlement window gating — is not sophisticated for its own sake. Each component exists to close a specific correctness gap. Removing any one of them reintroduces the gap it was designed to close.

The broader principle: in a distributed financial system, correctness is a property of the interaction between components, not of any individual component. Designing for correctness at the system level, not the service level, is the engineering discipline that separates platforms that hold up from platforms that require constant firefighting.

utm-git.github.io/utkarshmehrotra · linkedin.com/in/utkarshmehrotra- · github.com/utm-git